Topics Discussed

1. Process Concept
2. Process State
3. Process Control Block
4. Context Switch
5. Process Scheduling
6. Schedulers
7. CPU-bound and I/O -bound process
8. Operation on Process

1. Process Concept

- Program in execution.
- Need certain resources - such as CPU time, memory, files, and I/O devices.
- Resources are allocated to the process either when it is created or while it is executing.
- Program by itself is not a process;
- Program is a *passive* entity, such as the contents of a file stored on disk.
- Process is an *active* entity,

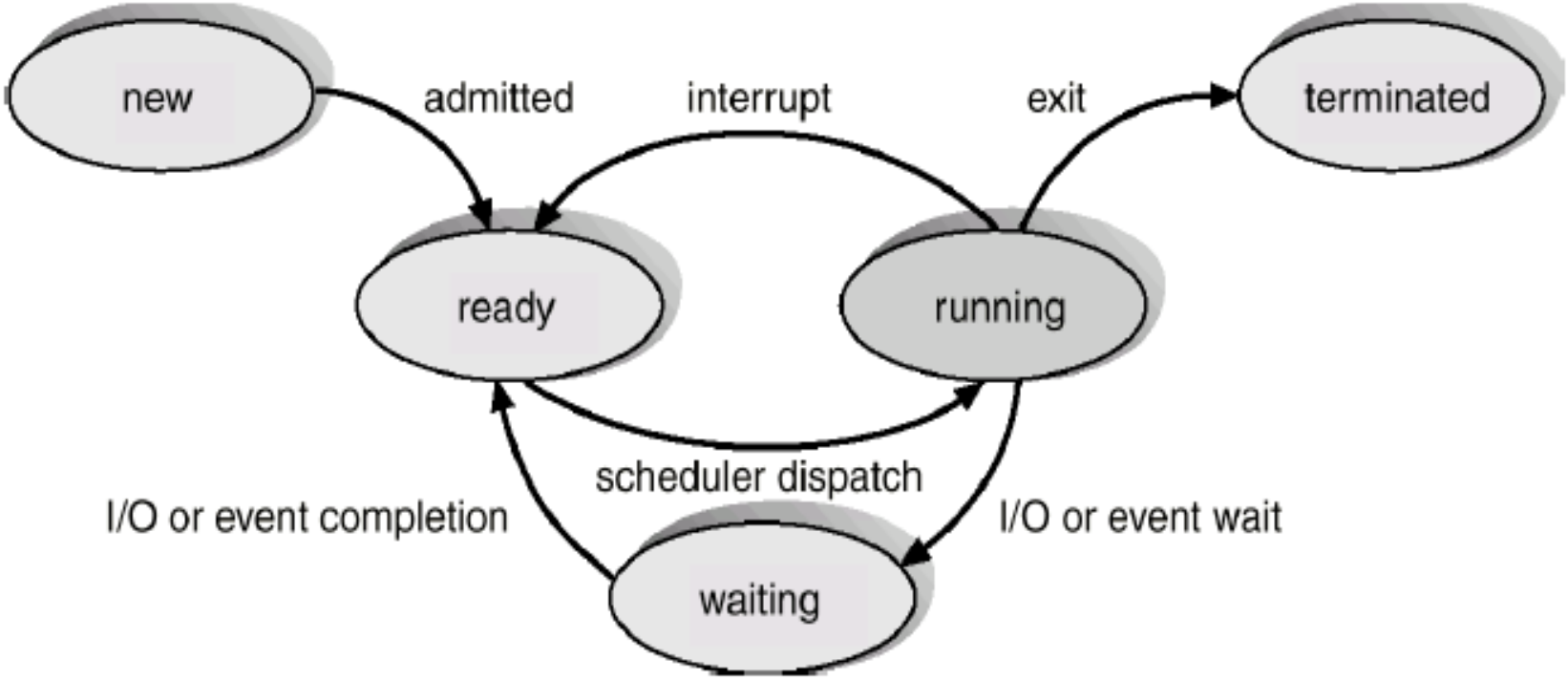
Process Concept

- System consists of a collection of processes:
  - OS processes execute system code
  - User processes execute user code.
- All these processes may execute concurrently.
- Single-user system:
  - User may be able to run several programs at one time
    - word processor, web browser, and e-mail package.
  - Eg: Microsoft Windows and Macintosh OS,

2. Process State

- As a process executes, it changes *state.*
- State: current activity of that process
  1. ***new:*** Process is being created.
  2. ***running:*** Instructions are being executed.
  3. ***waiting:*** Process is waiting for some event to occur.
  4. ***ready:*** Process is waiting to be assigned to a processor.
  5. ***terminated:*** Process has finished execution.

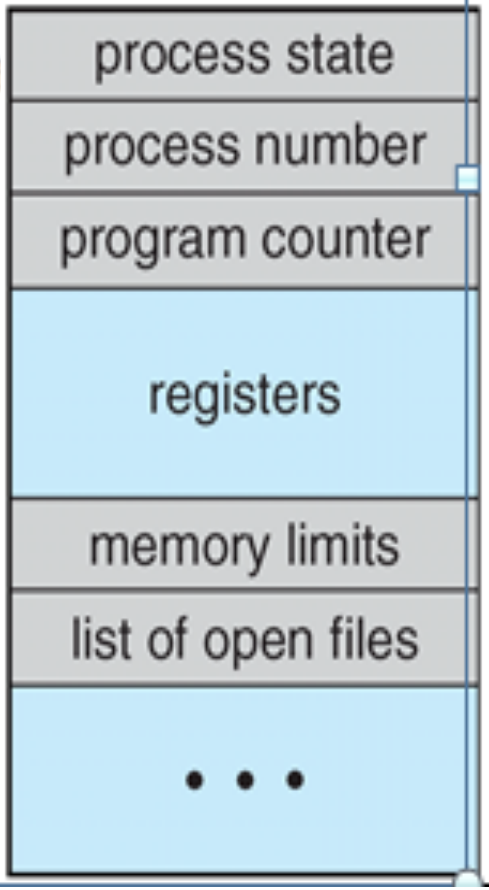
Diagram of Process State



2. Process Control Block

- Also called ***task control block.***
- Each process is represented in the OS.
- It contains some information associated with a

1. Process state
2. Program counter
3. CPU registers
4. CPU scheduling information
5. Memory-management information
6. Accounting information
7. I/O status information



Process Control Block (PCB) cont..

- **Process state:**
  - State may be new, ready, running, waiting, halted.
- **Program counter:**
  - Indicates the address of the next instruction to be executed.
- **CPU registers:**
  - Include accumulators, index registers, stack pointers, and general-purpose registers, plus condition-code information.
- **CPU-scheduling information:**
  - Includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

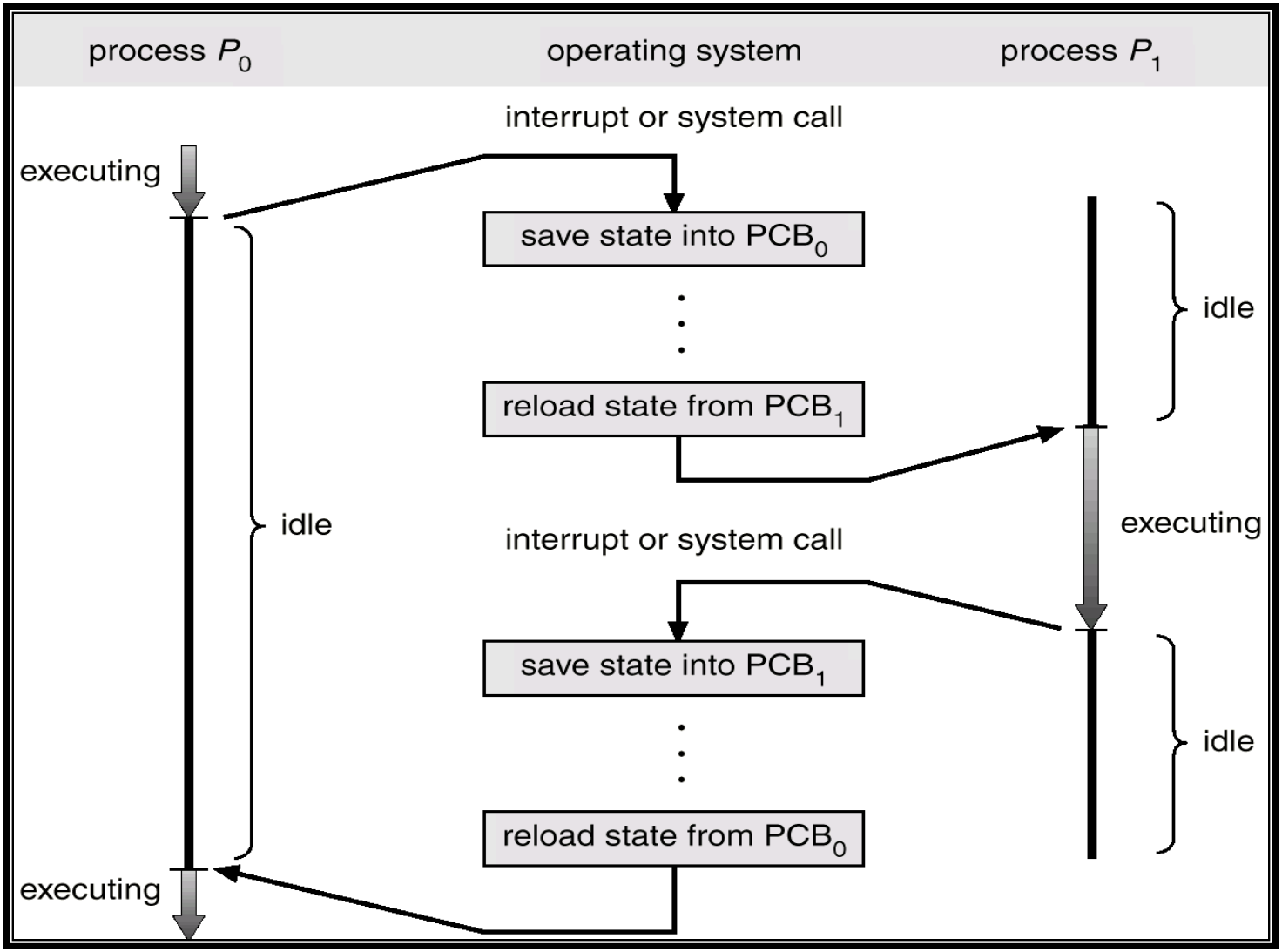
Process Control Block (PCB) cont..

- **Memory-management information:**
  - Include information of the base and limit registers, page tables, or segment tables.
- **Accounting information:**
  - Includes the amount of CPU time, time limits, account numbers, job or process numbers, and so on.
- **Status information:**
  - Includes list of I/O devices allocated to this process, a list of open files, and so on.

4. Context Switch

- When CPU switches to another process, the system must
  - save the state of the old process and
  - load the saved state for the new process.
- Context-switch time is overhead; the system does no useful work while switching.
- Its speed varies from machine to machine.
- Typical speed range from 1 to 1000 microseconds.
- Time dependent on hardware support.

CPU Switch From Process to Process



5. Process Scheduling

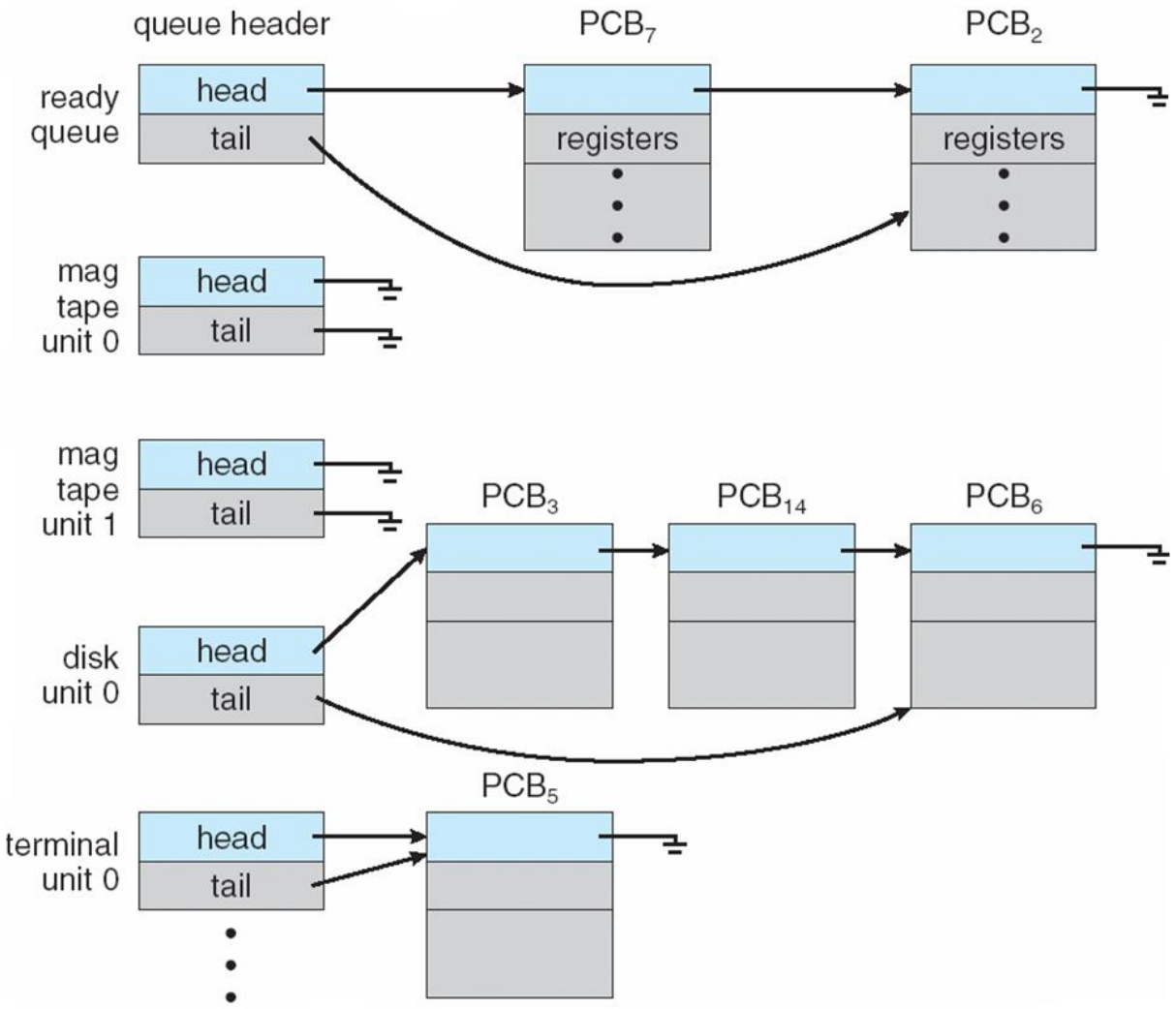
- CPU Scheduling Queues
  - ***Job queue***
    - set of all processes in the system.
  - ***Ready queue***
    - set of all processes residing in main memory, ready and waiting to execute.
  - ***Device queues***
    - set of processes waiting for an I/O device.
  - Process migrates between the various queues.
  - Queues- stored as a linked list.
  - Queue header contains 2 pointers to point first and final PCBs in the list.

Process Scheduling

- The objective of timesharing is to switch the CPU with each program while it is running
- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.

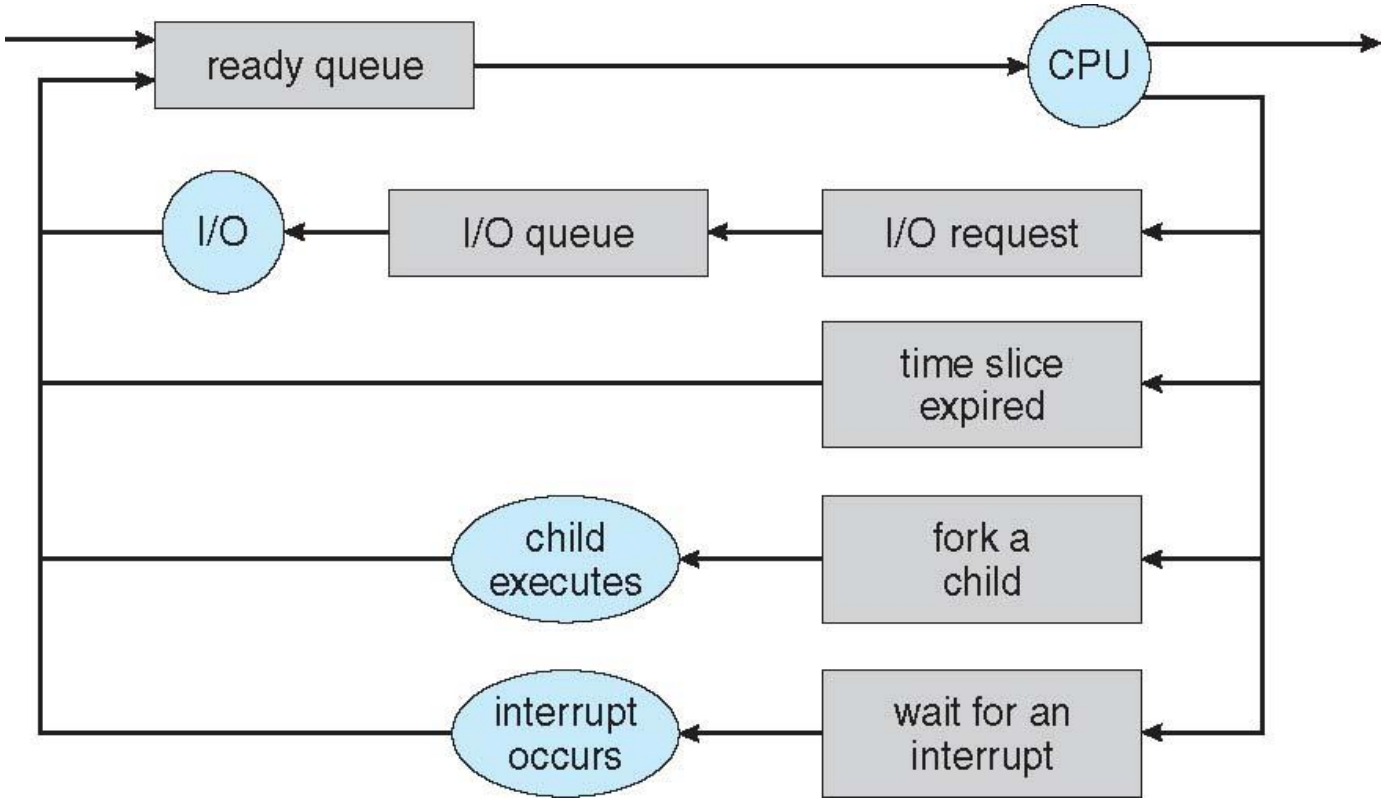
- To meet these objectives the process scheduler selects an available process(possible from a set of several available processes) for program execution on the CPU.

Ready Queue And Various I/O Device Queues



Representation of Process Scheduling

- **Queueing diagram** represents queues, resources, flows

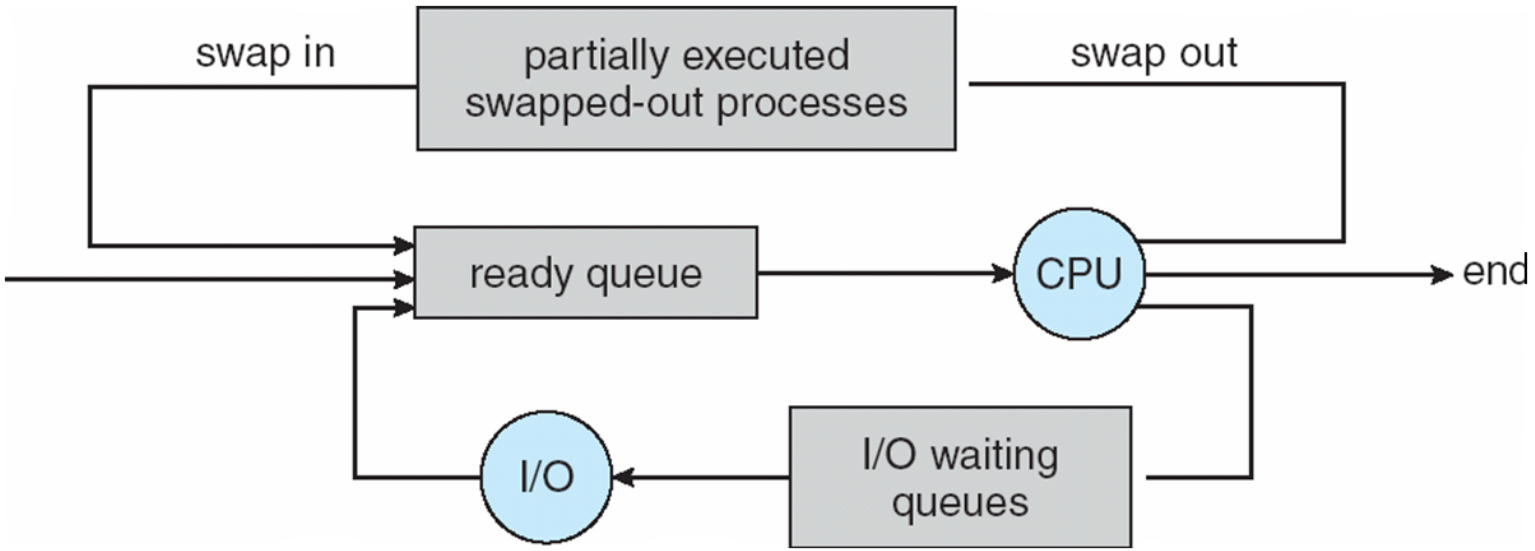


Schedulers

- ***Long-term scheduler (or job scheduler)***
  - Selects processes from job pool and loads them into memory for execution.
- ***Short-term scheduler (or CPU scheduler)***
  - Selects from among the processes that are ready to execute, and allocates the CPU to one of them.
- ***Medium-term scheduler (or swapping)***
  - removes processes from memory (and from CPU).
  - At some later time, the process can be reintroduced into memory and its execution can be continued where it left off.
  - time-sharing systems

Addition of Medium Term Scheduling

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**



Schedulers

- Difference between schedulers is the frequency of their execution.
- **Short-term scheduler**
  - Frequently select a new process for the CPU.
  - Fast.
- **Long-term scheduler**
  - Executes much less frequently.
  - Slow.
  - Controls the *degree of multiprogramming*.
  - Select a good process mix of I/O-bound and CPU-bound processes.
  - On some systems -may be absent or minimal.
    - Ex- time-sharing systems such as UNIX

7. CPU-bound and I/O-bound process

- Processes can be described as either:
  - **I/O-*bound process*** –
    - Spends more time doing I/O than computations.
    - If all processes are I/O bound, ready queue will always be empty.
    - Many short CPU bursts.
  - **CPU-*bound process***
    - Spends more time doing computations than I/O.
    - If all processes are CPU bound, I/O waiting queue will always be empty, devices will go unused.
    - Few very long CPU bursts.
- System with combination of CPU-bound and I/O-bound processes will give best performance.

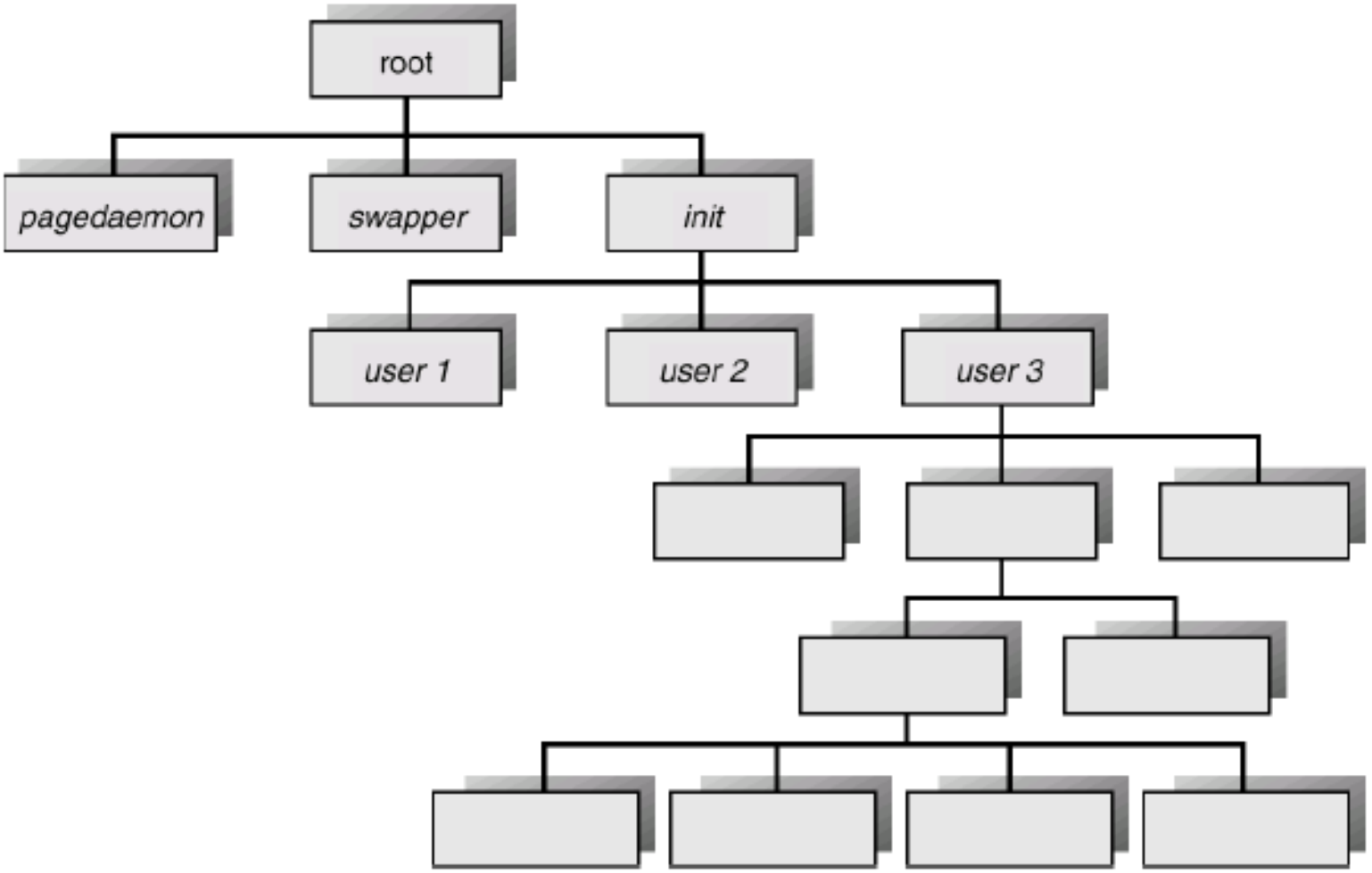
8. Operations on Processes

- Process Creation
- Process Termination

Process Creation

- Process - create several new processes, via a **create-process** system call.
- Creating process - **parent** process.
- New processes - **children** of that process.
- Each of these new processes - create other processes, forms a tree of processes.

A Tree of Processes on a UNIX System



Process Creation

When a process creates a new process

- **Two possibilities exist in terms of execution:**
  1. Parent continues to execute concurrently with its children.
  2. Parent waits until some or all of its children have terminated.
- **Two possibilities exist in terms of the address space:**
  1. Child process is a duplicate of the parent process.
  2. Child process has a program loaded into it.

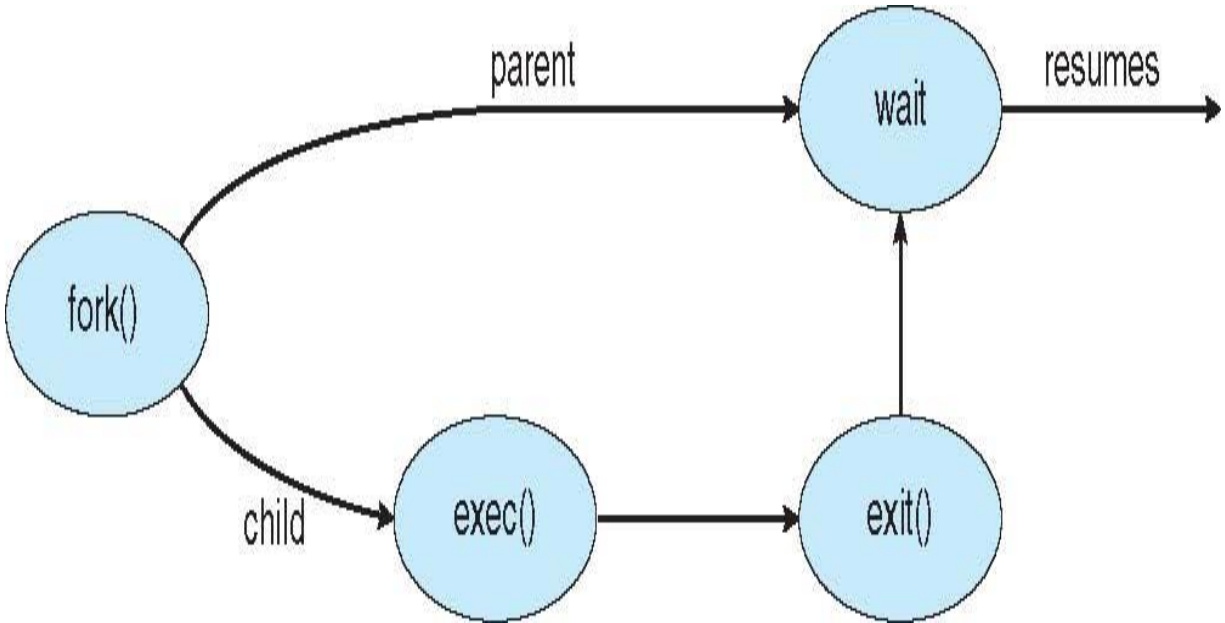
Process Creation (Cont.)

UNIX examples

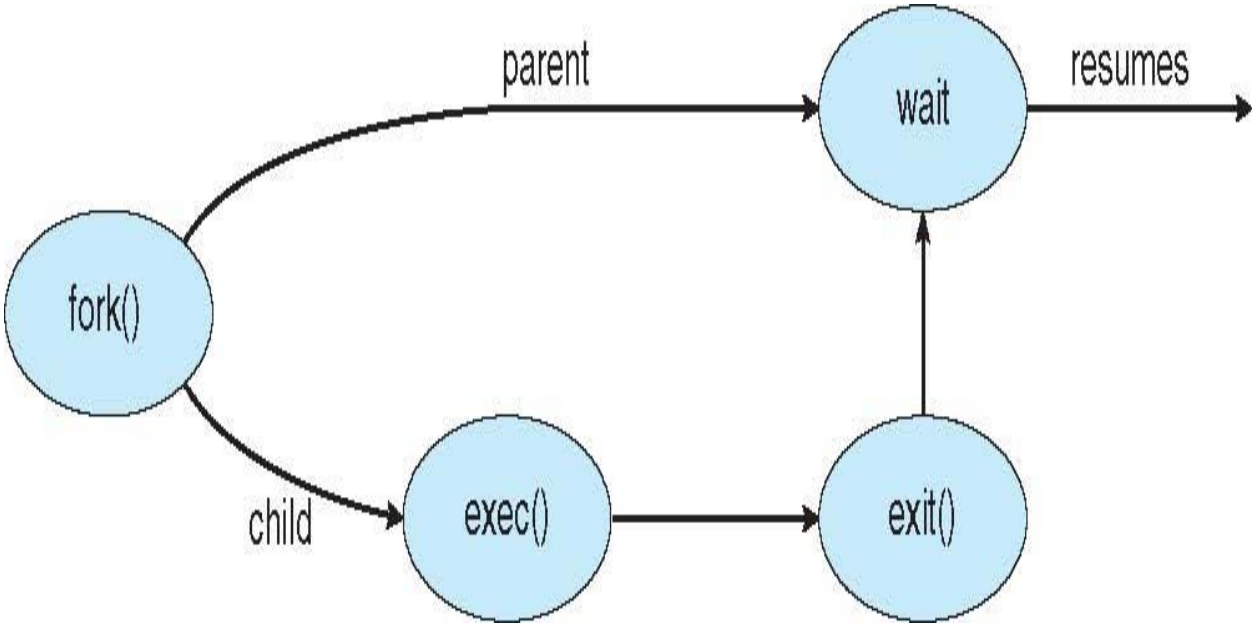
- **fork** system call creates new process
- **exec** system call used after a **fork** to replace the process' memory space with a new program.

Example: In UNIX,

- Each process is identified by its process ID, which is unique.
- **fork** system call is used to create new process.
- New process consists of a copy of the address space of the original process.
- This mechanism allows parent to communicate with its child.
- Both processes continue execution after **fork** system call, with one difference:
  - Return code for **fork** system call is zero for the new (child) process,
  - Process identifier of the child is returned to the parent.



- **execlp** system call
  - used after a **fork** system call by one of the two processes to replace the process' memory space with a new program.
  - loads a binary file into memory and starts its execution.
- **wait** system call
  - While the child runs, parent can issue a **wait** system call to move itself off the ready queue until the termination of the child.
  - Returns the process ID of a terminated child



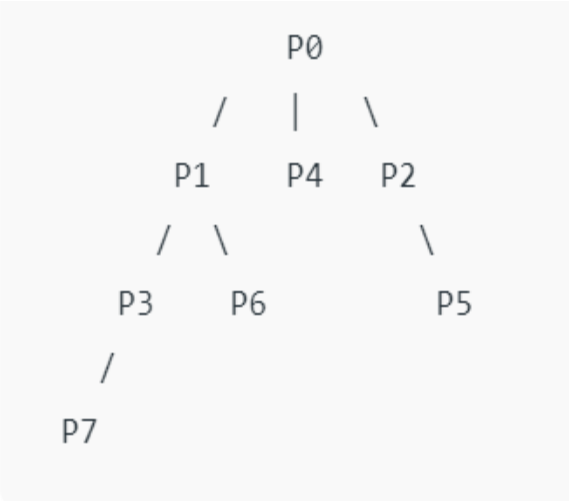
fork() system call

Number of processes: 2^n

Where n = no of fork calls

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
        fork();
        fork();
        fork();
        printf("hello\n");
        return 0;
}
```

- So there are total eight processes (new child processes and one original process).



- The main process: P0 Processes created by the 1st fork: P1 Processes created by the 2nd fork: P2, P3 Processes created by the 3rd fork: P4, P5, P6, P7

Process Creation (Cont.)

- Parent creates a child using **fork** system call.
- pid= zero for the child.
- pid= greater than zero for parent.
- Child overlays its address space using **execlp** system call.
- Parent waits for child to complete with **wait** system call.
- When child completes, parent completes using **exit** system call.

```
#include <stdio.h>

void main(int argc, char *argv[])
{
int pid;

    /* fork another process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       exit(-1);
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL) ;
       printf("Child Complete");
       exit(0) ;
```

Process Termination

A Process terminates

- When it executes last statement and asks the OS to delete it by using **exit** system call..
- Process may return output to its parent process via **wait** system call.
- Process's resources are deallocated by operating system.

Process Termination

- Parent may terminate execution of children using **abort** system call.
  - Child has exceeded allocated resources.
  - Task assigned to child is no longer required.
  - Parent is exiting.
    - OS does not allow child to continue if its parent terminates.
    - Cascading termination - initiated by the OS.